

Nombre: Nathaly Bravo

06/07/2026

Métodos Numéricos

Tarea 08

1) Dados los datos:

x_i : 4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1

y_i : 102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.72

a. Construya el polinomio por mínimos cuadrados de grado 1 y calcule el error

El polinomio de grado 1

```
In [10]: import numpy as np
import matplotlib.pyplot as plt
```

```
In [82]: def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:

    c_ind = sum(ys)

    c_1 = sum(xs)

    c_0 = len(xs)

    return (c_1, c_0, c_ind)

def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
    c_1 = 0
    c_0 = 0
    c_ind = 0
    for xi, yi in zip(xs, ys):

        c_ind += xi * yi
```

```

        c_1 += xi * xi

        c_0 += xi

    return (c_1, c_0, c_ind)

```

```

In [83]: xs = [4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1]

ys = [102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.7]

m,b = 71.61024372, -191.57241853
xs0 = np.linspace(-4, 12, 200)

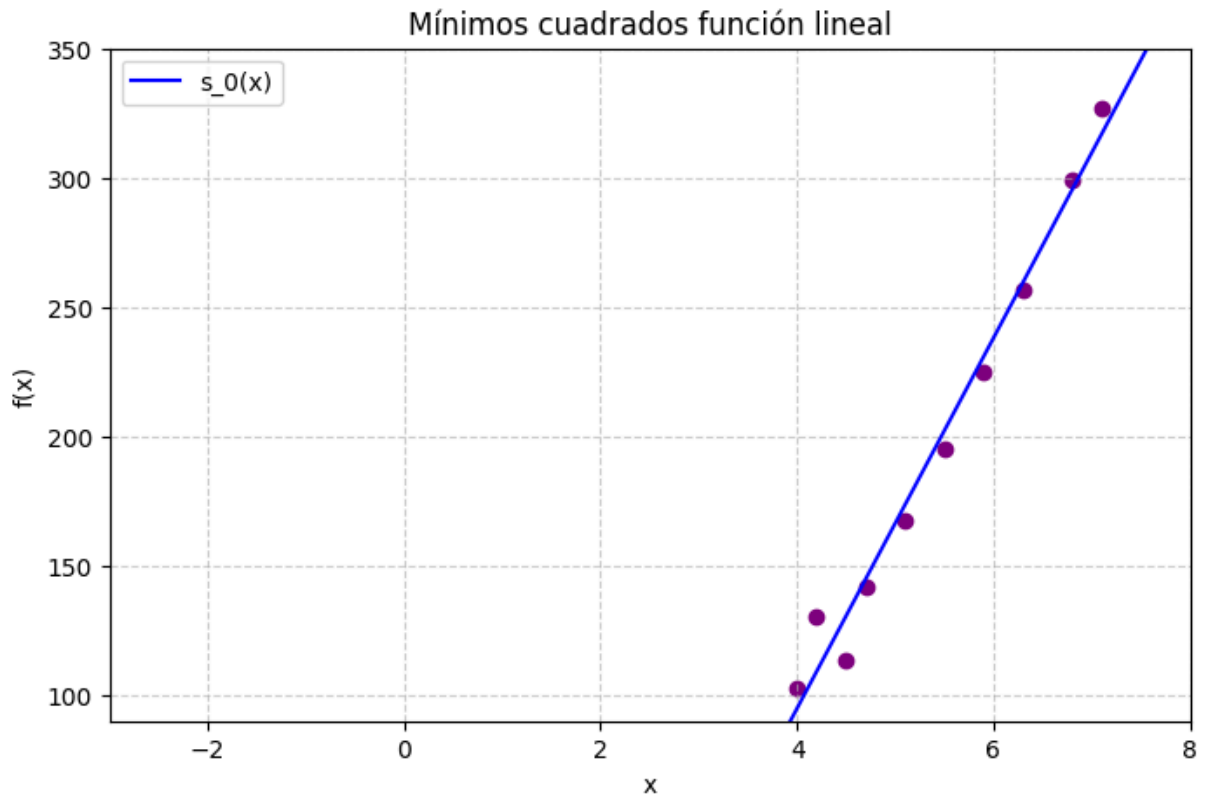
def s_0(x):
    return m * x + b

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='purple')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función lineal')
plt.legend()
plt.xlim(-3, 8)
plt.ylim(90, 350)
plt.show()

```



Para calcular el error:

```
In [84]: def error_minimos_cuadrados(xs, ys, a0, a1):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a0 + a1 * x
              error += (y - y_hat)**2
          return error
```

```
In [85]: E = error_minimos_cuadrados(xs, ys, b, m)
          print("Error:", E)
```

Error: 1058.8388862638901

b. Construya el polinomio por mínimos cuadrados de grado 2 y calcule el error.

El polinomio de grado 2 tiene la forma $ax^2 + bx + c$

```
In [86]: def der_parcial_2 (xs: list, ys: list) -> tuple[float, float, float]:
          c2 = c1 = c0 = c_ind = 0.0

          c_ind = sum(ys)
          c0 = len(xs)
          for xi in xs:
              c2 += xi**2
              c1 += xi
```

```

    return (c2, c1, c0, c_ind)

def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:
    c2 = c1 = c0 = c_ind = 0.0
    for xi, yi in zip(xs, ys):
        c2 += xi**3
        c1 += xi**2
        c0 += xi
        c_ind += xi * yi
    return (c2, c1, c0, c_ind)

def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
    c2 = c1 = c0 = c_ind = 0.0
    c_1 = 0
    c_0 = 0
    c_ind = 0
    for xi, yi in zip(xs, ys):
        c2 += xi**4
        c1 += xi**3
        c0 += xi **2
        c_ind += xi **2 * yi

    return (c2, c1, c0, c_ind)

```

In [87]:

```

def ajustar_min_cuadrados(xs, ys, derivadas):
    n = len(derivadas)

```

```

    A = []
    B = []
    for der in derivadas:
        coefs = der(xs, ys)
        A.append(list(coefs[:n]))
        B.append(coefs[n])

    A = np.array(A, dtype=float)
    B = np.array(B, dtype=float)

    # Resolver el sistema
    solucion = np.linalg.solve(A, B)
    return tuple(solucion)

```

In [88]:

```

xs = [4.0, 4.2, 4.5, 4.7, 5.1, 5.5 ,5.9, 6.3, 6.8, 7.1]

ys = [102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.7]

a,b,c = ajustar_min_cuadrados(xs, ys, [der_parcial_2, der_parcial_1, der_parcial_0])
xs0 = np.linspace(-4, 12, 200)

def s_0(x):
    return a * x**2 + b*x +c

```

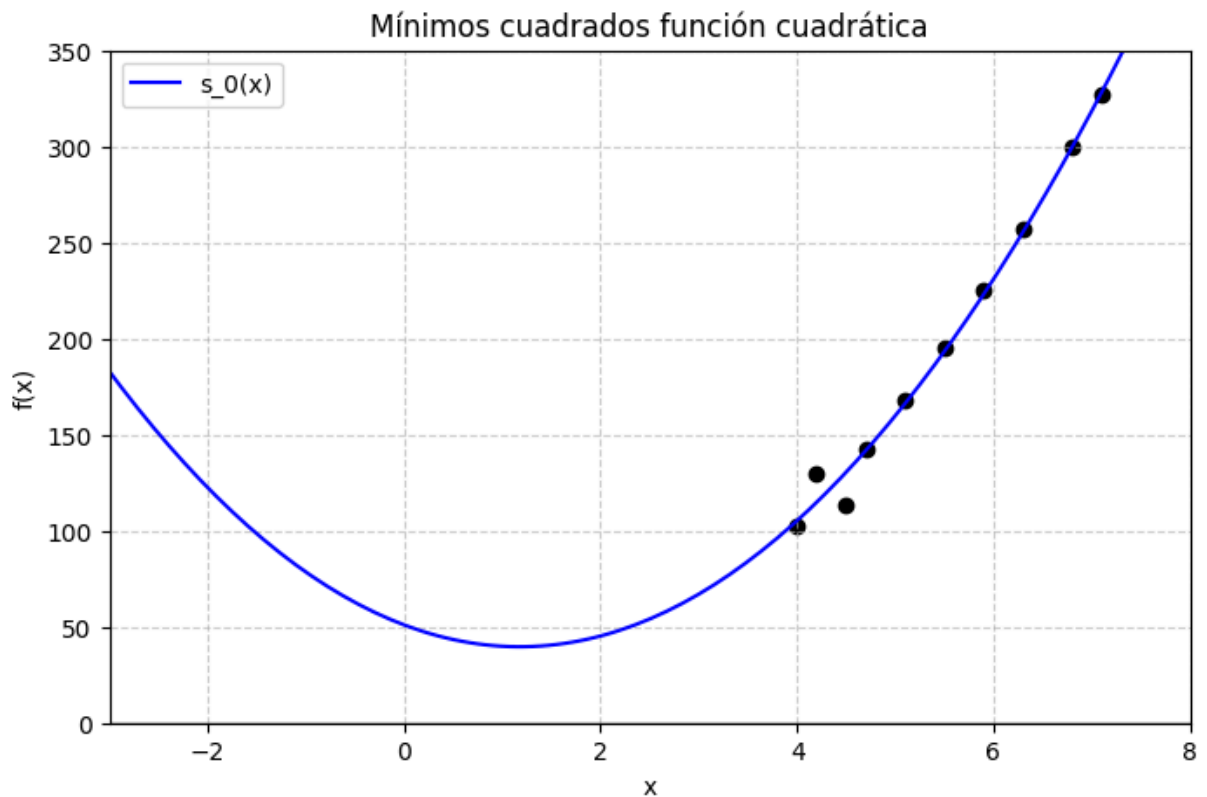
```

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='black')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función cuadrática')
plt.legend()
plt.xlim(-3, 8)
plt.ylim(0, 350)
plt.show()

```



Para calcular el error:

```

In [20]: def error_minimos_cuadrados(xs, ys, a, b, c):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a*x**2 + b*x + c
              error += (y - y_hat)**2
          return error

```

```

In [21]: E = error_minimos_cuadrados(xs, ys, a, b, c)
          print("Error:", E)

```

Error: 551.6562001170238

c. Construya el polinomio por mínimos cuadrados de grado 3 y calcule el error.

El polinomio de grado 3 tiene la forma $ax^3 + bx^2 + cx + d = 0$

```
In [22]: def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = 0.0
    c0 = len(xs)
    c_ind = sum(ys)

    for xi in xs:
        c3 += xi**3
        c2 += xi**2
        c1 += xi

    return (c3, c2, c1, c0, c_ind)

def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = c0 = c_ind = 0.0

    for xi, yi in zip(xs, ys):
        c3 += xi**4
        c2 += xi**3
        c1 += xi**2
        c0 += xi
        c_ind += xi * yi

    return (c3, c2, c1, c0, c_ind)

def der_parcial_2(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = c0 = c_ind = 0.0

    for xi, yi in zip(xs, ys):
        c3 += xi**5
        c2 += xi**4
        c1 += xi**3
        c0 += xi**2
        c_ind += (xi**2) * yi

    return (c3, c2, c1, c0, c_ind)

def der_parcial_3(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = c0 = c_ind = 0.0

    for xi, yi in zip(xs, ys):
        c3 += xi**6
        c2 += xi**5
        c1 += xi**4
        c0 += xi**3
        c_ind += (xi**3) * yi
```

```
return (c3, c2, c1, c0, c_ind)
```

```
In [23]: xs = [4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1]

ys = [102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.7]

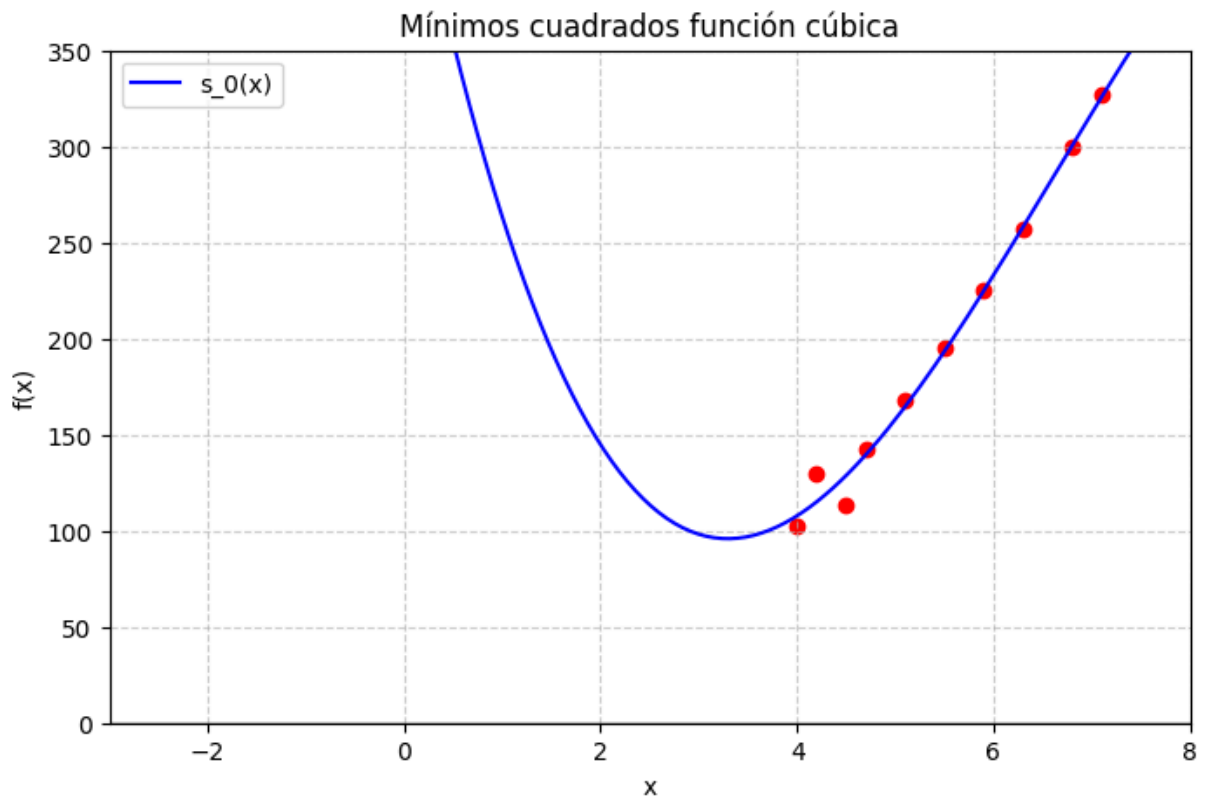
a,b,c,d = ajustar_min_cuadrados(xs, ys, [der_parcial_3, der_parcial_2, der_parcial_1])
xs0 = np.linspace(-4, 12, 200)

def s_0(x):
    return a * x**3 + b*x**2 + c*x + d

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='red')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función cúbica')
plt.legend()
plt.xlim(-3, 8)
plt.ylim(0, 350)
plt.show()
```



Para calcular el error

```
In [25]: def error_minimos_cuadrados(xs, ys, a, b, c, d):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a*x**3 + b*x**2 + c*x + d
              error += (y - y_hat)**2
          return error
```

```
In [26]: E = error_minimos_cuadrados(xs, ys, a, b, c, d)
          print("Error:", E)
```

Error: 518.3830647403066

d. Construya el polinomio por mínimos cuadrados de la forma be^{ax} y calcule el error.

El modelo no es lineal, pero se puede linealizar usando logaritmos, teniendo

$$y = be^{ax}$$

, aplicamos

$$\ln(y) = \ln(b) + ax$$

y definimos el cambio de variable

$$Y = \ln(y)$$

$$B = \ln(b)$$

$A = a$, así quedándonos

$$Y = Ax + B$$

In [90]: `import numpy as np`

```
xs = np.array(xs)
ys = np.array(ys)

Y = np.log(ys)
```

In [91]: `def der_parcial_A(xs, Y):`

```
    return (
        np.sum(xs**2),
        np.sum(xs),
        np.sum(xs * Y)
    )
```

`def der_parcial_B(xs, Y):`

```
    return (
        np.sum(xs),
        len(xs),
        np.sum(Y)
    )
```

In [92]: `A, B = ajustar_min_cuadrados(xs, Y, [der_parcial_A, der_parcial_B])`

In [93]: `xs = [4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1]`

```
ys = [102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.7
```

```
a = A
b = np.exp(B)
xs0 = np.linspace(-4, 12, 200)
```

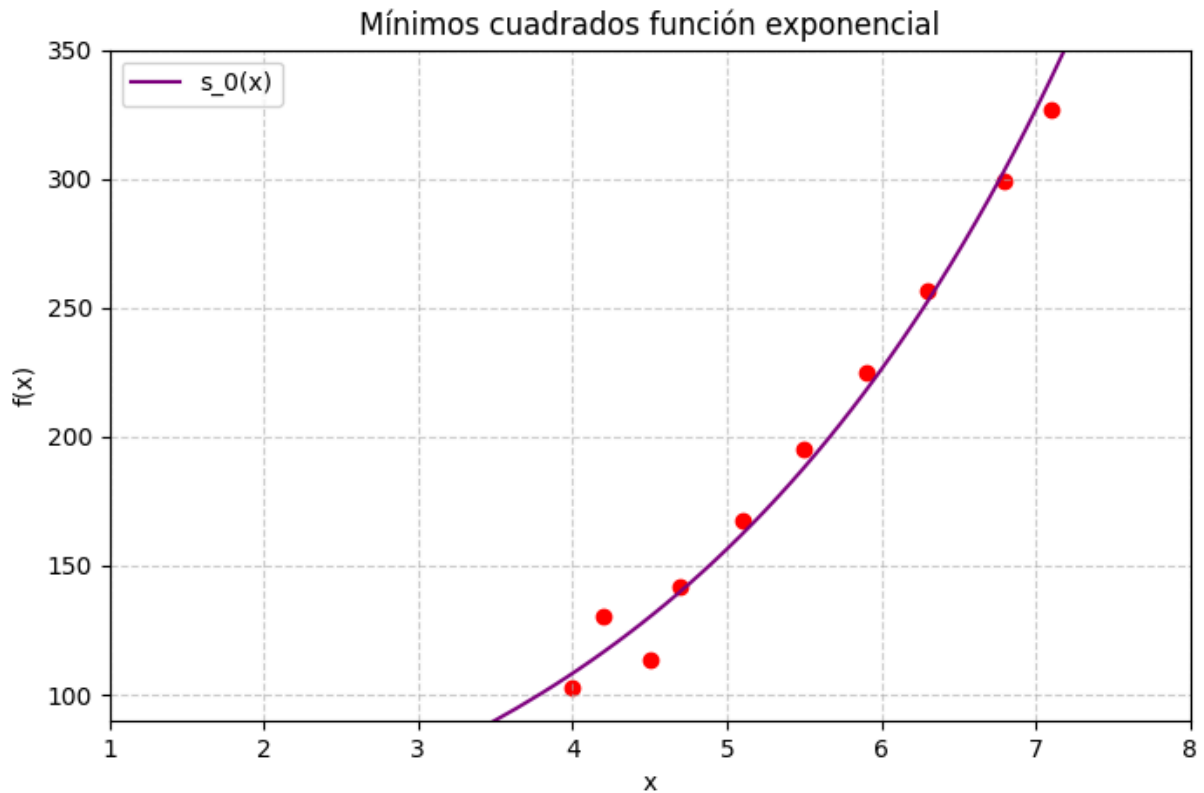
```
def s_0(x):
    return b*np.exp(a*x)
```

```
plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='red')
```

```
plt.plot(xs0, s_0(xs0), label='s_0(x)', color='purple')
```

```
plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
```

```
plt.title('Mínimos cuadrados función exponencial')
plt.legend()
plt.xlim(1, 8)
plt.ylim(90, 350)
plt.show()
```



Para calcular el error

```
In [32]: def error_minimos_cuadrados(xs, ys, a, b):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = b*np.exp(a*x)
              error += (y - y_hat)**2
          return error
```

```
In [33]: E = error_minimos_cuadrados(xs, ys, a,b)
          print("Error:", E)
```

Error: 821.0051092577766

e. Construya el polinomio por mínimos cuadrados de la forma bx^a y calcule el error.

Teniendo

$$y = bx^a$$

, aplicamos logaritmo, quedándonos

$$\ln(y) = \ln(b) + \ln(x^a)$$

realizando el cambio de variable queda

$$Y = \ln(y)$$

$$X = \ln(x)$$

$$B = \ln(b)$$

$$Y = aX + B$$

```
In [34]: X = np.log(xs)
Y = np.log(ys)

def der_parcial_a(X, Y):
    return (
        np.sum(X**2),
        np.sum(X),
        np.sum(X * Y)
    )

def der_parcial_B(X, Y):
    return (
        np.sum(X),
        len(X),
        np.sum(Y)
    )

a, B = ajustar_min_cuadrados(X, Y, [der_parcial_a, der_parcial_B])

b = np.exp(B)
```

```
In [35]: xs = [4.0, 4.2, 4.5, 4.7, 5.1, 5.5, 5.9, 6.3, 6.8, 7.1]
ys = [102.56, 130.11, 113.18, 142.05, 167.53, 195.14, 224.87, 256.73, 299.50, 326.7]

xs0 = np.linspace(1, 12, 200)

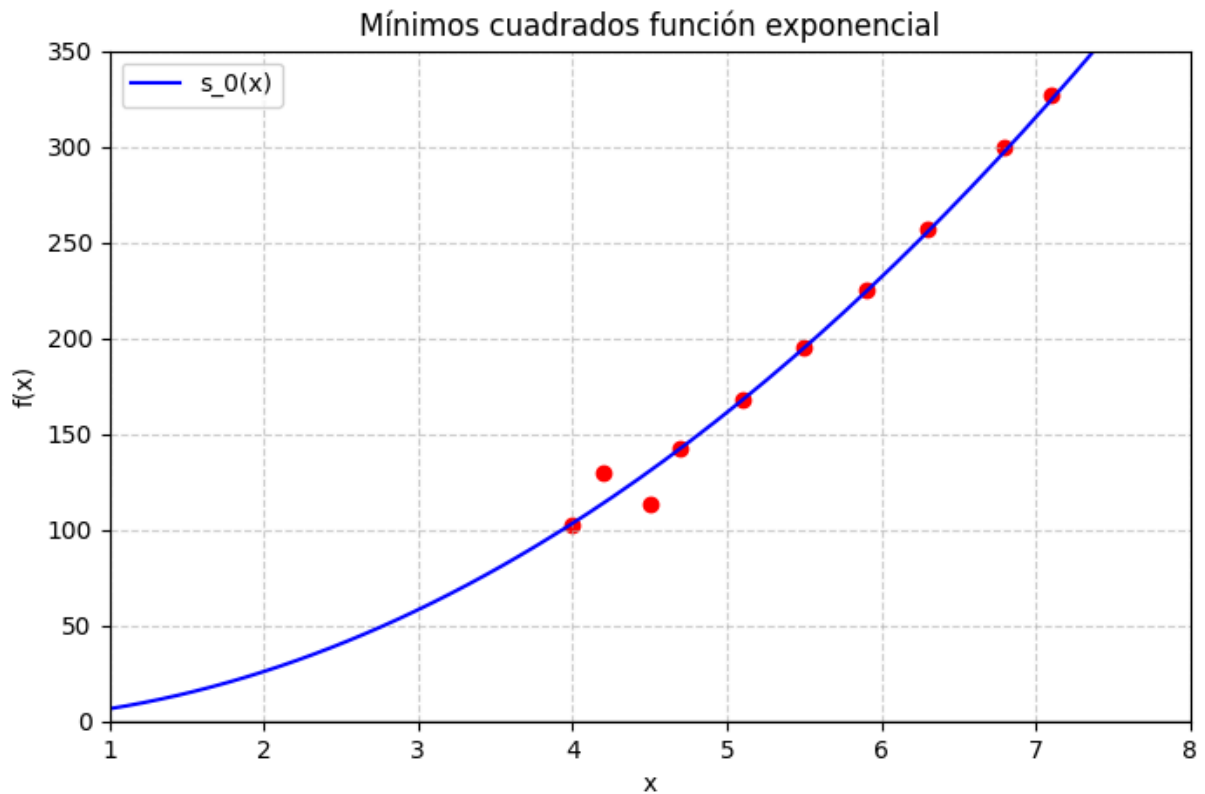
def s_0(x):
    return b*(x**a)

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='red')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
```

```
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función exponencial')
plt.legend()
plt.xlim(1, 8)
plt.ylim(0, 350)
plt.show()
```



Para calcular el error

```
In [37]: def error_minimos_cuadrados(xs, ys, a, b):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = b*x**a
              error += (y - y_hat)**2
          return error
```

```
In [38]: E = error_minimos_cuadrados(xs, ys, a,b)
          print("Error:", E)
```

Error: 581.5572726013454

2) Repita el ejercicio 5 para los siguientes datos

x_i : 0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6

y_i : 0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015

a)

```
In [80]: def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:
```

```
    c_ind = sum(ys)
```

```
    c_1 = sum(xs)
```

```
    c_0 = len(xs)
```

```
    return (c_1, c_0, c_ind)
```

```
def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
```

```
    c_1 = 0
```

```
    c_0 = 0
```

```
    c_ind = 0
```

```
    for xi, yi in zip(xs, ys):
```

```
        c_ind += xi * yi
```

```
        c_1 += xi * xi
```

```
        c_0 += xi
```

```
    return (c_1, c_0, c_ind)
```

```
In [81]: xs = [0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6]
```

```
ys = [0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015]
```

```
m,b = ajustar_min_cuadrados(xs, ys, [der_parcial_1, der_parcial_0])
```

```
xs0 = np.linspace(-1, 2, 200)
```

```
def s_0(x):
```

```
    return m * x + b
```

```
plt.figure(figsize=(8, 5))
```

```
for xi, yi in zip(xs, ys):
```

```
    plt.scatter([xi], [yi], color='red')
```

```
plt.plot(xs0, s_0(xs0), label='s_0(x)', color='black')
```

```
plt.axhline(0, color='black', linewidth=1)
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.xlabel('x')
```

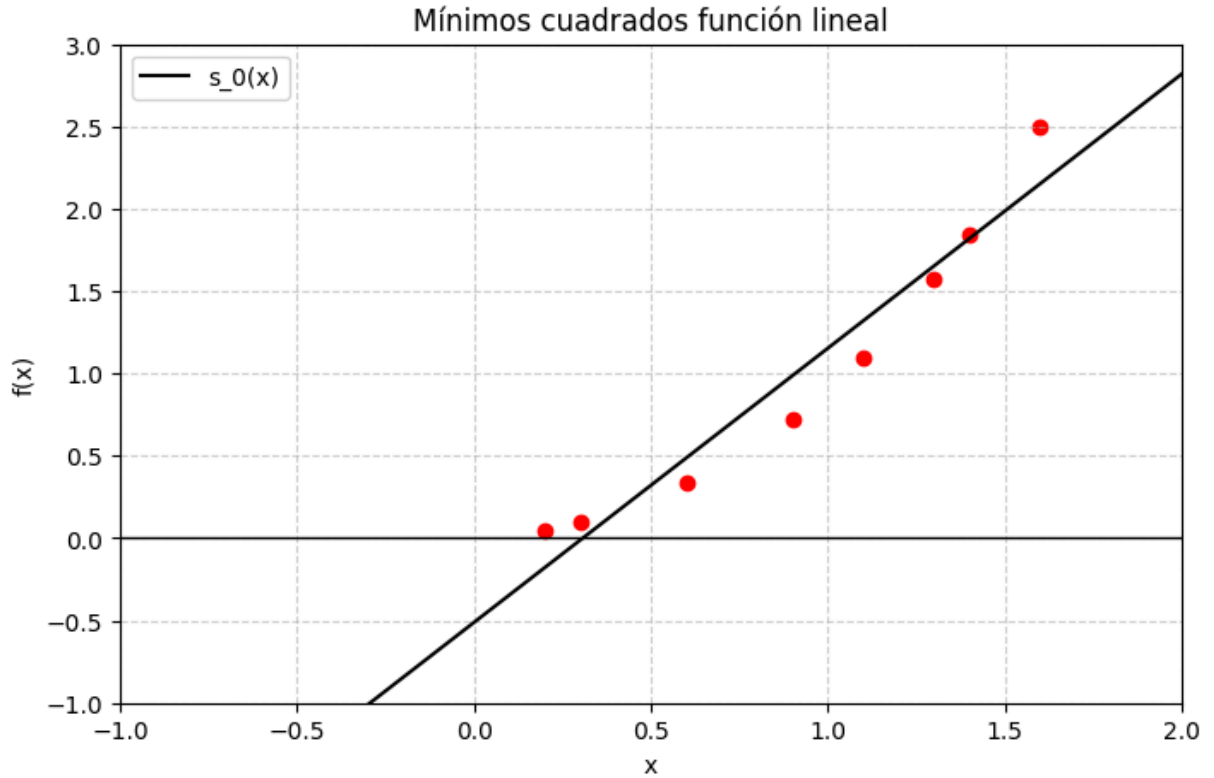
```
plt.ylabel('f(x)')
```

```
plt.title('Mínimos cuadrados función lineal')
```

```
plt.legend()
```

```
plt.xlim(-1, 2)
```

```
plt.ylim(-1, 3)
plt.show()
```



Para calcular el error

```
In [42]: def error_minimos_cuadrados(xs, ys, a0, a1):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a0 + a1 * x
              error += (y - y_hat)**2
          return error
```

```
In [43]: E = error_minimos_cuadrados(xs, ys, b, m)
          print("Error:", E)
```

Error: 0.3355898557594882

b)

```
In [78]: def der_parcial_2 (xs: list, ys: list) -> tuple[float, float, float]:
          c2 = c1 = c0 = c_ind = 0.0

          c_ind = sum(ys)
          c0 = len(xs)
          for xi in xs:
              c2 += xi**2
              c1 += xi

          return (c2, c1, c0, c_ind)
```

```

def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:
    c2 = c1 = c0 = c_ind = 0.0
    for xi, yi in zip(xs, ys):
        c2 += xi**3
        c1 += xi**2
        c0 += xi
        c_ind += xi * yi
    return (c2, c1, c0, c_ind)

def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
    c2 = c1 = c0 = c_ind = 0.0
    c_1 = 0
    c_0 = 0
    c_ind = 0
    for xi, yi in zip(xs, ys):
        c2 += xi**4
        c1 += xi**3
        c0 += xi **2
        c_ind += xi **2 * yi

    return (c2, c1, c0, c_ind)

```

```

In [79]: xs = [0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6]

ys = [0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015]

a,b,c = ajustar_min_cuadrados(xs, ys, [der_parcial_2,der_parcial_1, der_parcial_0])
xs0 = np.linspace(-1, 2, 200)

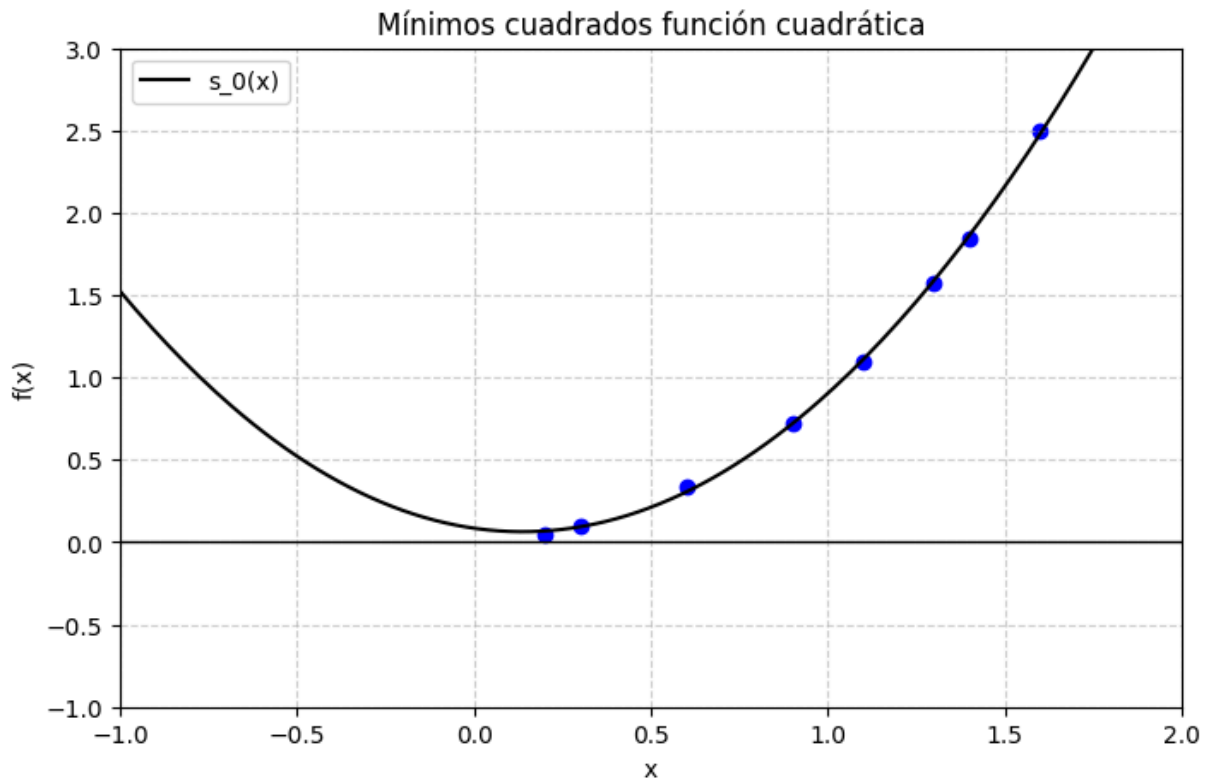
def s_0(x):
    return a*x**2 + b*x +c

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='blue')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='black')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función cuadrática')
plt.legend()
plt.xlim(-1, 2)
plt.ylim(-1, 3)
plt.show()

```



Para calcular el error

```
In [46]: def error_minimos_cuadrados(xs, ys, a, b, c):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a*x**2 + b*x + c
              error += (y - y_hat)**2
          return error
```

```
In [47]: E = error_minimos_cuadrados(xs, ys, a, b, c)
          print("Error:", E)
```

Error: 0.0024199149294266927

c)

```
In [76]: def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float, float, float]:
          c3 = c2 = c1 = 0.0
          c0 = len(xs)
          c_ind = sum(ys)

          for xi in xs:
              c3 += xi**3
              c2 += xi**2
              c1 += xi

          return (c3, c2, c1, c0, c_ind)

          def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float, float, float]:
```



```

c3 = c2 = c1 = c0 = c_ind = 0.0

for xi, yi in zip(xs, ys):
    c3 += xi**4
    c2 += xi**3
    c1 += xi**2
    c0 += xi
    c_ind += xi * yi

return (c3, c2, c1, c0, c_ind)

def der_parcial_2(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = c0 = c_ind = 0.0

    for xi, yi in zip(xs, ys):
        c3 += xi**5
        c2 += xi**4
        c1 += xi**3
        c0 += xi**2
        c_ind += (xi**2) * yi

    return (c3, c2, c1, c0, c_ind)

def der_parcial_3(xs: list, ys: list) -> tuple[float, float, float, float, float]:
    c3 = c2 = c1 = c0 = c_ind = 0.0

    for xi, yi in zip(xs, ys):
        c3 += xi**6
        c2 += xi**5
        c1 += xi**4
        c0 += xi**3
        c_ind += (xi**3) * yi

    return (c3, c2, c1, c0, c_ind)

```

```

In [77]: xs = [0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6]

ys = [0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015]

a,b,c, d = ajustar_min_cuadrados(xs, ys, [der_parcial_3, der_parcial_2, der_parcial_1])
xs0 = np.linspace(-1, 2, 200)

def s_0(x):
    return a*x**3 + b*x**2 + c*x + d

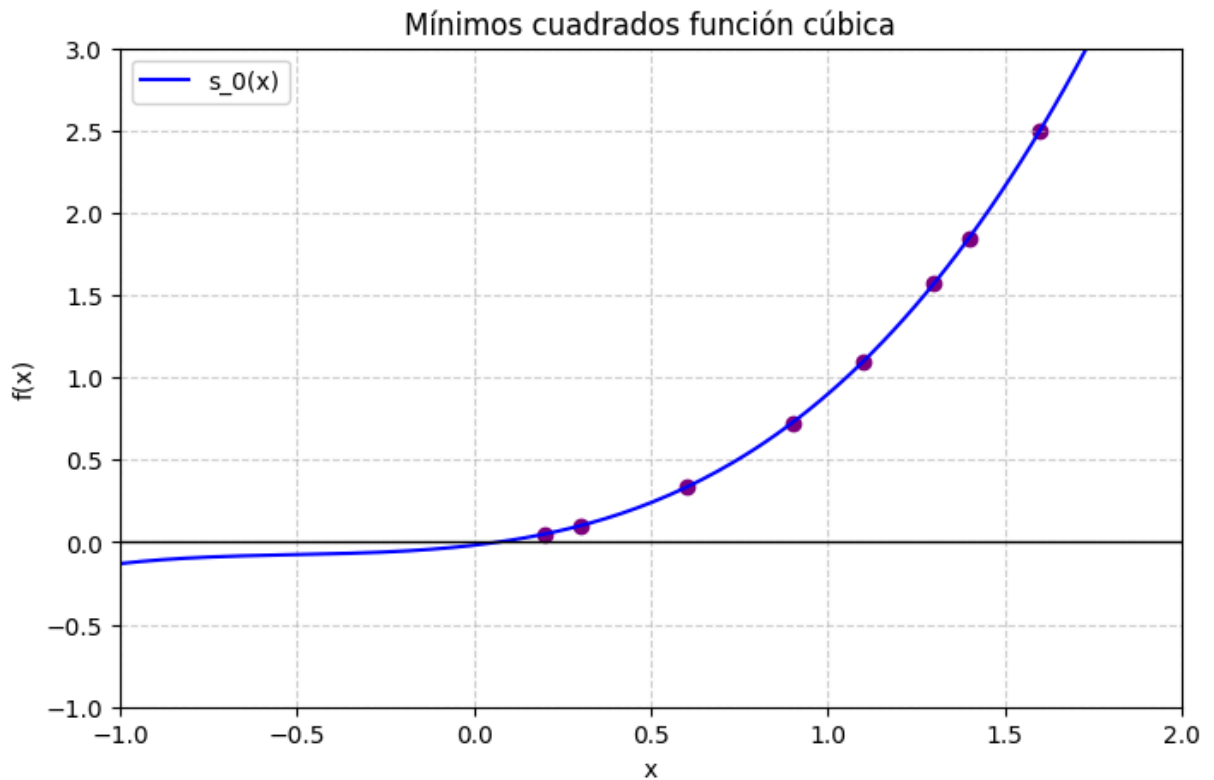
plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='purple')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)

```

```
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función cúbica')
plt.legend()
plt.xlim(-1, 2)
plt.ylim(-1, 3)
plt.show()
```



Para calcular el error

```
In [50]: def error_minimos_cuadrados(xs, ys, a, b, c, d):
          error = 0.0
          for x, y in zip(xs, ys):
              y_hat = a*x**3 + b*x**2 + c*x + d
              error += (y - y_hat)**2
          return error
```

```
In [51]: E = error_minimos_cuadrados(xs, ys, a, b, c, d)
          print("Error:", E)
```

Error: 5.074671555627565e-06

d)

```
In [70]: xs = np.array(xs)
          ys = np.array(ys)

          Y = np.log(ys)
```

```

def der_parcial_A(xs, Y):
    return (
        np.sum(xs**2),
        np.sum(xs),
        np.sum(xs * Y)
    )

def der_parcial_B(xs, Y):
    return (
        np.sum(xs),
        len(xs),
        np.sum(Y)
    )

A, B = ajustar_min_cuadrados(xs, Y, [der_parcial_A, der_parcial_B])

xs = [0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6]

ys = [0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015]

a = A
b = np.exp(B)
xs0 = np.linspace(-4, 12, 200)

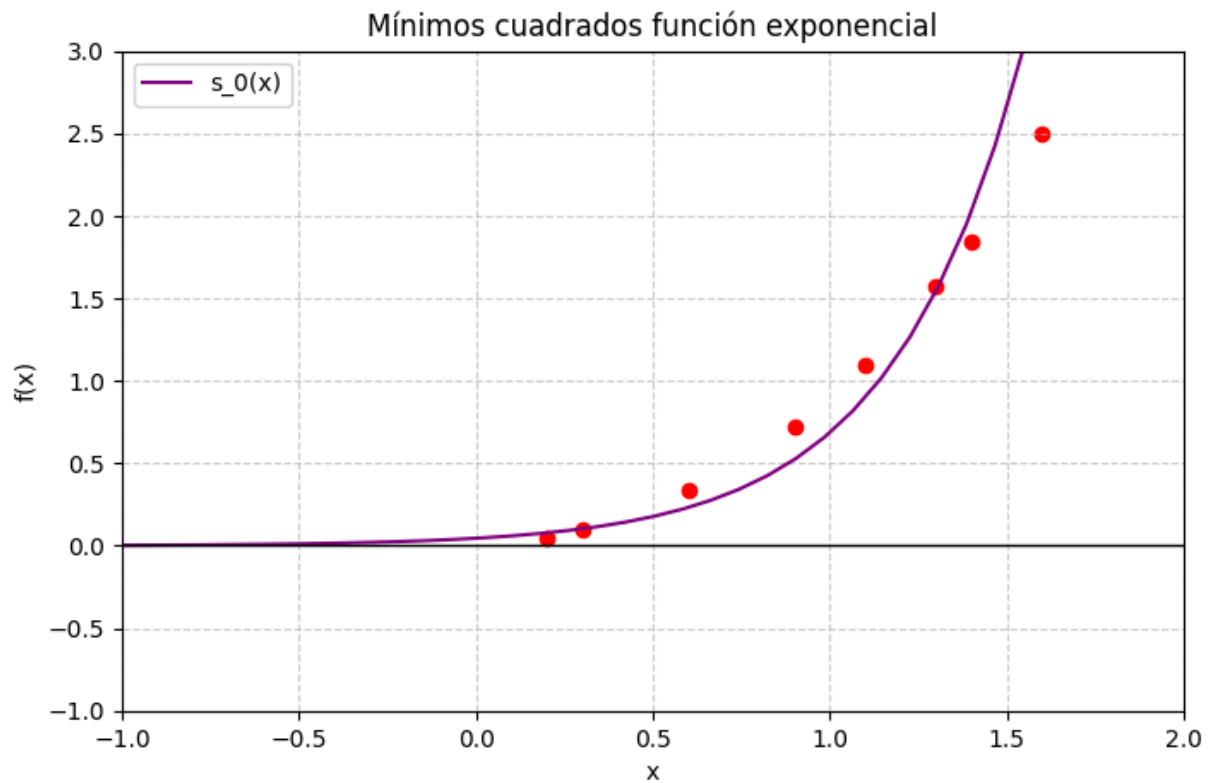
def s_0(x):
    return b*np.exp(a*x)

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='red')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='purple')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función exponencial')
plt.legend()
plt.xlim(-1, 2)
plt.ylim(-1, 3)
plt.show()

```



Para calcular el error

```
In [53]: def error_minimos_cuadrados(xs, ys, a, b):
            error = 0.0
            for x, y in zip(xs, ys):
                y_hat = b*np.exp(a*x)
                error += (y - y_hat)**2
            return error
```

```
In [54]: E = error_minimos_cuadrados(xs, ys, a, b)
          print("Error:", E)
```

Error: 1.075048503189181

e)

```
In [55]: X = np.log(xs)
Y = np.log(ys)

def der_parcial_a(X, Y):
    return (
        np.sum(X**2),
        np.sum(X),
        np.sum(X * Y)
    )

def der_parcial_B(X, Y):
    return (
        np.sum(X),
        len(X),
```

```

        np.sum(Y)
    )

a, B = ajustar_min_cuadrados(X, Y, [der_parcial_a, der_parcial_B])

b = np.exp(B)

```

```

In [69]: xs = [0.2, 0.3, 0.6, 0.9, 1.1, 1.3, 1.4, 1.6]

ys = [0.050446, 0.098426, 0.33277, 0.72660, 1.0972, 1.5697, 1.8487, 2.5015]

xs0 = np.linspace(-1, 12, 200)

def s_0(x):
    return b*(x**a)

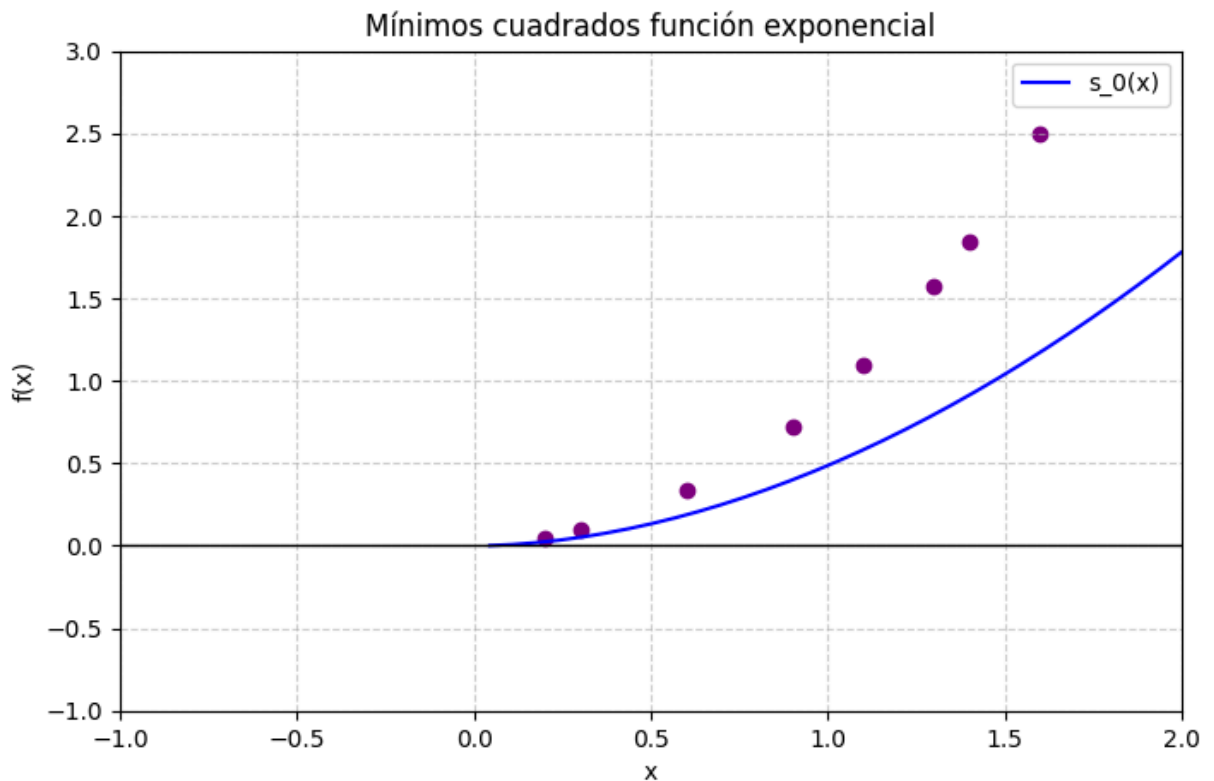
plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='purple')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función exponencial')
plt.legend()
plt.xlim(-1, 2)
plt.ylim(-1, 3)
plt.show()

```

C:\Users\Nathaly\AppData\Local\Temp\ipykernel_28232\1759998175.py:10: RuntimeWarning: invalid value encountered in power
 return b*(x**a)



3) La siguiente tabla muestra los promedios de puntos del colegio de 20 especialistas en matemáticas y ciencias computacionales, junto con las calificaciones que recibieron estos estudiantes en la parte de matemáticas de la prueba ACT (Programa de Pruebas de Colegios Americanos) mientras estaban en secundaria. Grafique estos datos y encuentre la ecuación de la recta por mínimos cuadrados para estos datos.

28 3.84 29 3.75 25 3.21 28 3.65 28 3.23 27 3.87 27 3.63 29 3.75 28 3.75 21 1.66 33 3.20 28 3.12 28 3.41 28 2.96 29 3.38 26 2.92 23 3.53 30 3.10 27 2.03 24 2.81

```
In [68]: xs = [28, 25, 28, 27, 28, 33, 28, 29, 23, 27, 29, 28, 27, 29, 21, 28, 28, 26, 30, 24]
ys = [3.84, 3.21, 3.23, 3.63, 3.75, 3.2, 3.41, 3.38, 3.53, 2.03, 3.75, 3.65, 3.87, 3.75, 1.66, 3.12, 3.41, 2.96, 3.38, 2.92, 3.53, 3.10, 2.03, 2.81]

xs0 = np.linspace(20, 35, 200)

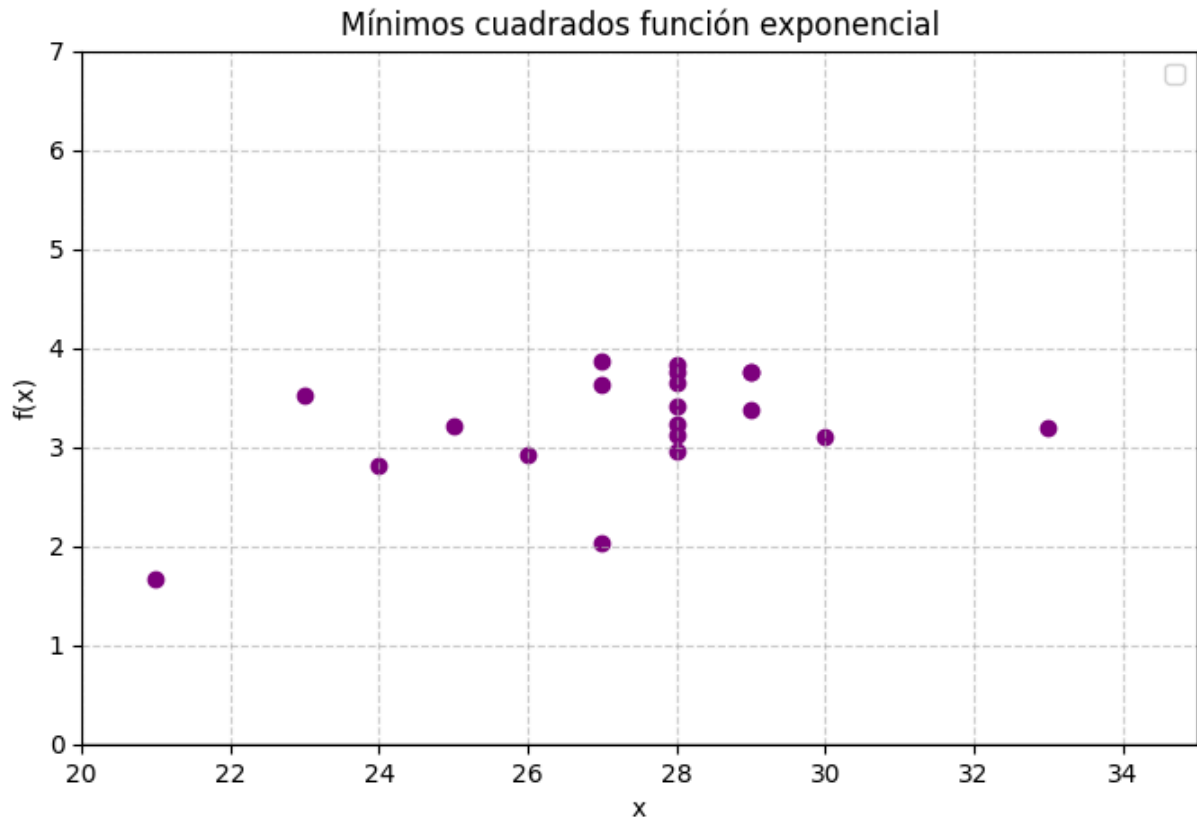
def s_0(x):
    return b*(x**a)

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='purple')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
```

```
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función exponencial')
plt.legend()
plt.xlim(20, 35)
plt.ylim(0, 7)
plt.show()
```

C:\Users\Nathaly\AppData\Local\Temp\ipykernel_28232\3802778383.py:23: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



Como no hay cambios muy bruscos, la mejor opción es un modelo lineal

```
In [ ]: def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:

    c_ind = sum(ys)

    c_1 = sum(xs)

    c_0 = len(xs)

    return (c_1, c_0, c_ind)

def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
    c_1 = 0
    c_0 = 0
    c_ind = 0
```

```

for xi, yi in zip(xs, ys):

    c_ind += xi * yi

    c_1 += xi * xi

    c_0 += xi

return (c_1, c_0, c_ind)

```

```

In [67]: xs = [28, 25, 28,27,28,33,28,29,23,27,29,28,27,29,21,28,28,26,30,24]
ys = [3.84,3.21,3.23,3.63,3.75,3.2,3.41,3.38,3.53,2.03,3.75,3.65,3.87,3.75,1.66,3.1

m,b = ajustar_min_cuadrados(xs, ys, [der_parcial_1, der_parcial_0])
xs0 = np.linspace(20, 35, 200)

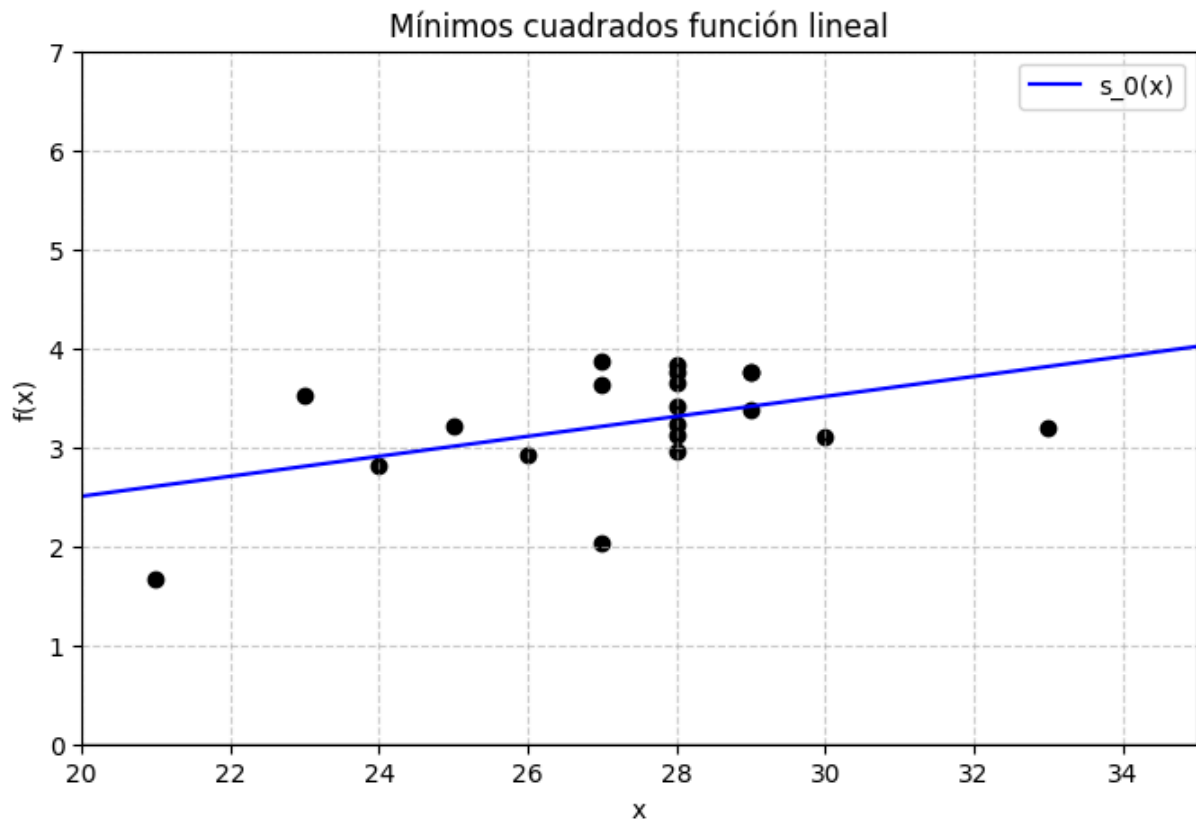
def s_0(x):
    return m*x+b

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='black')

plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función lineal')
plt.legend()
plt.xlim(20, 35)
plt.ylim(0, 7)
plt.show()

```

4. El siguiente conjunto de datos, presentado al Subcomité Antimonopolio del Senado, muestra las características comparativas de supervivencia durante un choque de automóviles de diferentes clases. Encuentre la recta por mínimos cuadrados que aproxima estos datos (la tabla muestra el porcentaje de vehículos que participaron en un accidente en los que la lesión más grave fue fatal o seria).

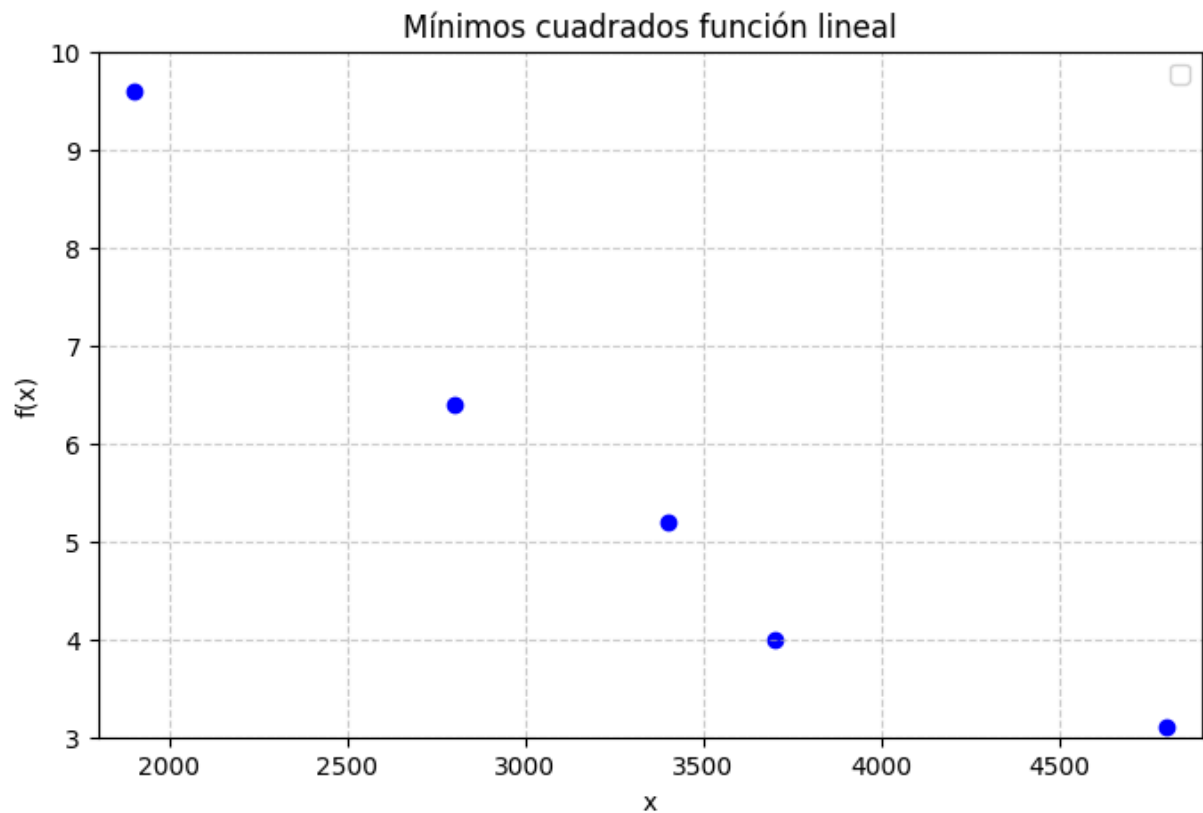
```
In [61]: xs = [4800, 3700, 3400, 2800, 1900]
         ys = [3.1, 4.0, 5.2, 6.4, 9.6]
```

```
In [66]: xs0 = np.linspace(1800, 4900, 400)

plt.figure(figsize=(8, 5))
for xi, yi in zip(xs, ys):
    plt.scatter([xi], [yi], color='blue')

plt.axhline(0, color='black', linewidth=1)
plt.grid(True, linestyle='--', alpha=0.6)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Mínimos cuadrados función lineal')
plt.legend()
plt.xlim(1800, 4900)
plt.ylim(3, 10)
plt.show()
```

C:\Users\Nathaly\AppData\Local\Temp\ipykernel_28232\4197200285.py:17: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



In [63]: `def der_parcial_1(xs: list, ys: list) -> tuple[float, float, float]:`

```
    c_ind = sum(ys)
```

```
    c_1 = sum(xs)
```

```
    c_0 = len(xs)
```

```
    return (c_1, c_0, c_ind)
```

```
def der_parcial_0(xs: list, ys: list) -> tuple[float, float, float]:
```

```
    c_1 = 0
```

```
    c_0 = 0
```

```
    c_ind = 0
```

```
    for xi, yi in zip(xs, ys):
```

```
        c_ind += xi * yi
```

```
        c_1 += xi * xi
```

```

        c_0 += xi

    return (c_1, c_0, c_ind)

```

```

In [65]: xs = [4800, 3700, 3400, 2800, 1900]
        ys = [3.1, 4.0, 5.2, 6.4, 9.6]

        m,b = ajustar_min_cuadrados(xs, ys, [der_parcial_1, der_parcial_0])
        xs0 = np.linspace(1800, 4900, 400)

        def s_0(x):
            return m*x + b

        plt.figure(figsize=(8, 5))
        for xi, yi in zip(xs, ys):
            plt.scatter([xi], [yi], color='black')

        plt.plot(xs0, s_0(xs0), label='s_0(x)', color='blue')
        plt.axhline(0, color='black', linewidth=1)
        plt.grid(True, linestyle='--', alpha=0.6)
        plt.xlabel('x')
        plt.ylabel('f(x)')
        plt.title('Mínimos cuadrados función lineal')
        plt.legend()
        plt.xlim(1800, 4900)
        plt.ylim(3, 10)
        plt.show()

```

